

SIMATS ENGINEERING
ASSESSMENT TOOL 1
ARTIFICIAL INTELLIGENCE
Course Code: CSA1718

Student Name	M NIHANTH
Register Number	192411241
Department	CSE
Date	14-07-2026

Contents

1. Water Jug Problem
2. Mars Rover as an Intelligent Agent
3. Eight-Queens Problem
4. OLA Cab Booking Problem-Solving Agent
5. Uniform Cost Search for Logistics Network

Prepared according to the questions and rubric provided in Assessment Tool

1.

1. Water Jug Problem

Problem Understanding

Two unmarked jugs are available: a 4-gallon jug and a 3-gallon jug. The allowed operations are filling a jug, emptying a jug, and pouring water from one jug into the other. The objective is to obtain exactly 2 gallons in the 4-gallon jug.

State Representation

Represent a state as (x, y) , where x is the amount of water in the 4-gallon jug and y is the amount in the 3-gallon jug.

Initial state: $(0, 0)$	Goal state: $(2, y)$
---	--

Solution Process

Step	Action	4-gallon jug	3-gallon jug
1	Start with both jugs empty	0	0
2	Fill the 3-gallon jug	0	3
3	Pour the 3-gallon jug into the 4-gallon jug	3	0
4	Fill the 3-gallon jug again	3	3
5	Pour into the 4-gallon jug until it becomes full	4	2
6	Empty the 4-gallon jug	0	2
7	Pour the remaining 2 gallons into the 4-gallon jug	2	0

State Sequence and Final Result

$$(0,0) \rightarrow (0,3) \rightarrow (3,0) \rightarrow (3,3) \rightarrow (4,2) \rightarrow (0,2) \rightarrow (2,0)$$

Therefore, the 4-gallon jug contains exactly 2 gallons, and the goal is achieved.

2. Mars Rover as an Intelligent Agent

A Mars rover is an autonomous intelligent agent that senses the Martian environment, plans movements, avoids hazards, collects samples and sends scientific data to Earth.

i. Percepts of the Agent

- Images from navigation and scientific cameras
- Distance and location of rocks, craters and obstacles
- Soil and rock composition
- Temperature, pressure, wind and radiation levels
- Ground slope, terrain roughness and wheel slippage
- Current position, orientation and travelled distance
- Battery level, solar-energy availability and internal system temperature
- Communication signals and commands from Earth
- Health status of motors, sensors and scientific instruments

ii. Characterisation of the Operating Environment

Property	Characterisation	Reason
Observability	Partially observable	The rover cannot observe the entire environment at one time.
Determinism	Stochastic	Terrain, dust and wheel slip create uncertain outcomes.
Task structure	Sequential	A present action changes future states and available actions.
Change	Dynamic	Lighting, temperature, dust and terrain conditions can change.
State values	Mostly continuous	Position, speed, time and temperature vary continuously.

Agents	Mainly single-agent	The rover acts independently while receiving guidance from Earth.
Knowledge	Partially unknown	It may encounter unseen rocks, slopes and hazards.
Accessibility	Inaccessible	Complete and exact information about Mars is unavailable.

iii. Actions the Agent Can Take

- Move forward or backward; turn left or right; stop
- Choose another route and avoid obstacles
- Capture and transmit photographs
- Extend the robotic arm and drill into rocks
- Collect, store and analyse soil or rock samples
- Measure atmospheric and environmental conditions
- Receive instructions and transmit scientific data
- Recharge batteries or enter power-saving mode
- Diagnose faults and recover from minor failures

iv. Performance Evaluation

Performance Measure	Evaluation
Scientific value	Quality and number of useful samples and measurements collected.
Navigation accuracy	Ability to reach selected destinations safely.
Obstacle avoidance	Success in detecting and avoiding hazards.
Energy efficiency	Useful scientific work completed per unit of energy consumed.
Mission completion	Percentage of assigned objectives successfully completed.

Safety and reliability	Protection of wheels, instruments and communication systems.
Data quality	Accuracy and usefulness of images and scientific results.
Time efficiency	Completion of planned tasks within available mission time.
Communication efficiency	Successful transmission of data with minimal loss.
Energy Efficiency = Useful Scientific Work Completed / Energy Consumed	

v. Most Suitable Agent Architecture

The most suitable design is a hybrid model-based, goal-based, utility-based and learning agent.

Component	Purpose
Model-based	Maintains an internal map of obstacles, visited areas, targets and safe routes.
Goal-based	Plans actions to reach scientific targets and complete mission objectives.
Utility-based	Balances scientific value, travel time, energy use and risk.
Learning	Improves navigation by learning from terrain images, wheel slips and previous outcomes.
Utility: $U = w_1S - w_2E - w_3T - w_4R$	

Here, S is scientific value, E is energy consumption, T is travel time, R is risk, and the w values represent their importance. This hybrid design is suitable because the rover must operate autonomously under uncertainty while conserving energy and maximising scientific value.

3. Eight-Queens Problem

The task is to place eight queens on an 8×8 chessboard so that no two queens attack each other. Therefore, no two queens may share the same row, column or diagonal.

Problem Components

Component	Formulation
State space	A partial arrangement represented as $[r_1, r_2, \dots, r_k]$, where r_i is the row of the queen in column i .
Initial state	An empty board: $[\]$.
Actions	Place one queen in the next column in a row that is not attacked.
Transition model	Append a safe row to the current state.
Goal test	Eight queens are placed with no shared row, column or diagonal.
Path cost	Each placement has cost 1; every complete solution has total cost 8.

Safety Conditions

For a new queen at row r and column c , and an existing queen at row r_i and column c_i :

$$r \neq r_i \quad \text{and} \quad |r - r_i| \neq |c - c_i|$$

The first condition prevents a shared row. The second prevents a shared diagonal.

Search-Space Analysis

Without constraints, the search space is $8^8 = 16,777,216$ possible assignments. Restricting the board to one queen per row and column reduces it to $8! = 40,320$ arrangements. Backtracking reduces the practical search space further by rejecting unsafe partial states immediately.

Suitable Search Strategy

Depth-First Search with Backtracking is suitable. Queens are placed one column at a time. When no safe position exists, the algorithm returns to the previous column and changes an earlier decision.

```
PROCEDURE SolveEightQueens
    board[1..8] ← 0

    IF PlaceQueen(board, 1) = TRUE THEN
        PRINT board
    ELSE
        PRINT "No solution exists"
    END IF
END PROCEDURE

FUNCTION PlaceQueen(board, column)
    IF column > 8 THEN
        RETURN TRUE
    END IF

    FOR row ← 1 TO 8 DO
        IF IsSafe(board, row, column) = TRUE THEN
            board[column] ← row

            IF PlaceQueen(board, column + 1) = TRUE THEN
                RETURN TRUE
            END IF

            board[column] ← 0
        END IF
    END FOR

    RETURN FALSE
END FUNCTION

FUNCTION IsSafe(board, newRow, newColumn)
    FOR oldColumn ← 1 TO newColumn - 1 DO
        oldRow ← board[oldColumn]

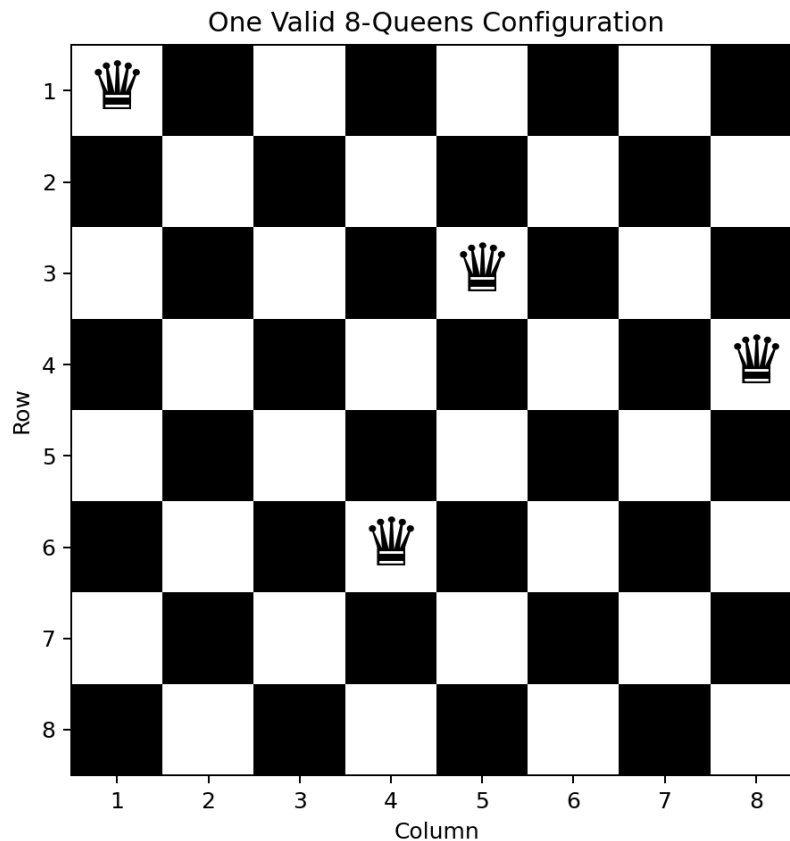
        IF oldRow = newRow THEN
            RETURN FALSE
        END IF

        IF ABS(oldRow - newRow) =
            ABS(oldColumn - newColumn) THEN
            RETURN FALSE
        END IF
    END FOR
END FUNCTION
```

```
RETURN TRUE  
END FUNCTION
```

One Valid Solution

[1, 5, 8, 6, 3, 7, 2, 4]



The displayed arrangement contains one queen in every row and column, and no two queens share a diagonal.

4. OLA Cab Booking Problem-Solving Agent

Problem Understanding

The customer provides a pickup location and destination. The mobile application must search for available cabs, compare cab types such as Micro, Mini, Sedan, Prime, Shared and SUV, and recommend a suitable option based on fare, comfort, waiting time and travel time.

Type of Agent

A goal-based and utility-based problem-solving agent is most suitable.

- The goal-based component ensures that the passenger reaches the destination.
- The utility-based component compares several valid cab options and selects the most beneficial one.

$$U(c) = w_1C + w_2R - w_3F - w_4W - w_5T$$

C = comfort, R = driver rating, F = fare, W = waiting time and T = travel time.

Problem Formulation

Element	Description
Initial state	Customer is at the pickup location and no cab is selected.
Goal state	Customer reaches the destination safely and conveniently.
State space	Locations, available cabs, drivers, routes, traffic, fare and estimated time.
Actions	Enter locations, select cab type, confirm/cancel booking, pay and track the trip.
Transition model	Searching → cab assigned → driver arriving → trip started → destination reached.
Path cost	A weighted combination of fare, travel time and waiting time.

Goal test	Current location equals destination and trip status is completed.
$\text{Cost} = \alpha(\text{Fare}) + \beta(\text{Travel Time}) + \gamma(\text{Waiting Time})$	

Pseudocode

```

PROCEDURE BookOLACab
  INPUT pickupLocation, destination
  INPUT passengerCount, comfortPreference, maximumBudget

  IF pickupLocation = destination THEN
    PRINT "Pickup and destination cannot be the same"
    STOP
  END IF

  routes ← FindRoutes(pickupLocation, destination)

  IF routes is empty THEN
    PRINT "No route is available"
    STOP
  END IF

  bestRoute ← FindMinimumCostRoute(routes)
  cabTypes ← {Micro, Mini, Sedan, Prime, Shared, SUV}
  availableOptions ← empty list

  FOR each cab IN cabTypes DO
    IF CabIsAvailable(cab, pickupLocation) THEN
      fare ← CalculateFare(cab, bestRoute)
      waitingTime ← EstimateWaitingTime(cab)
      utility ← CalculateUtility(cab, fare,
                                waitingTime, bestRoute.time)

      IF fare ≤ maximumBudget AND
         CabCapacity(cab) ≥ passengerCount THEN
        ADD cab and utility TO availableOptions
      END IF
    END IF
  END FOR

  IF availableOptions is empty THEN
    PRINT "No suitable cab is available"
    STOP
  END IF

  SORT availableOptions BY utility IN descending order
  DISPLAY availableOptions

```

```
INPUT selectedCab
```

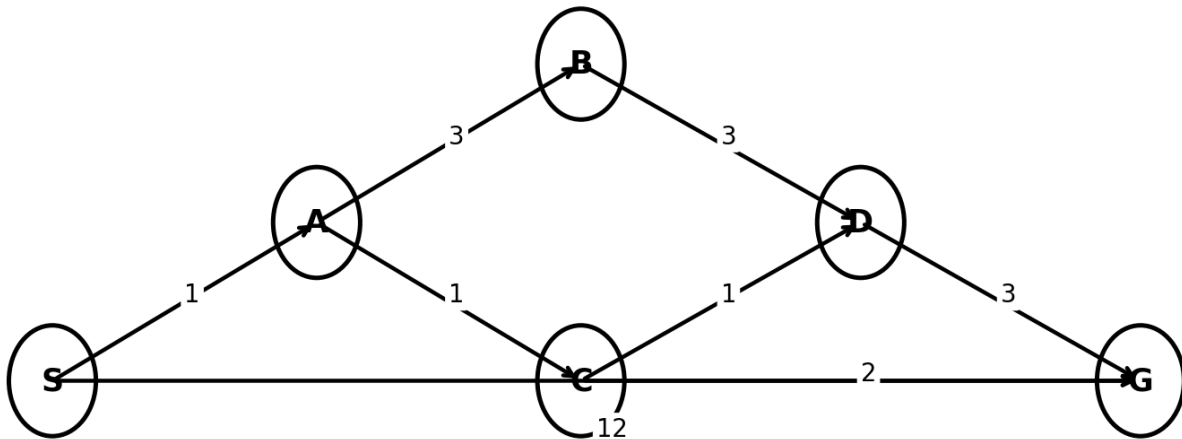
```
IF CustomerConfirms(selectedCab) THEN  
    driver ← AssignNearestDriver(selectedCab)  
    ConfirmBooking(driver, selectedCab, bestRoute)  
    TrackTripUntilDestination()  
    ProcessPayment()  
    PRINT "Trip completed successfully"  
ELSE  
    PRINT "Booking cancelled"  
END IF  
END PROCEDURE
```

This agent reaches the required destination while selecting the best combination of comfort, price, availability, driver rating and journey duration.

5. Uniform Cost Search for the Logistics Network

The objective is to identify the least-cost path from warehouse S to warehouse G. Uniform Cost Search expands the frontier node with the smallest cumulative path cost.

Weighted Delivery Network for Uniform Cost Search



Given Route Costs

Route	Cost
$S \rightarrow A$	1
$S \rightarrow G$	12
$A \rightarrow B$	3
$A \rightarrow C$	1
$B \rightarrow D$	3
$C \rightarrow D$	1
$C \rightarrow G$	2
$D \rightarrow G$	3

Why UCS Is Suitable

UCS orders nodes by cumulative cost $g(n)$. Because all route costs are positive, the algorithm is complete and optimal and will return the minimum-cost route.

Step-by-Step Expansion

Step	Node Expanded	Result	Priority Queue
1	Expand S	Add A with cost 1 and G with cost 12.	[(A,1), (G,12)]
2	Expand A	Add B with cost 4 and C with cost 2.	[(C,2), (B,4), (G,12)]
3	Expand C	Add D with cost 3 and update G to cost 4.	[(D,3), (B,4), (G,4)]
4	Expand D	Route to G costs 6, so retain existing G cost 4.	[(B,4), (G,4)]
5	Expand remaining cost-4 nodes	B gives a more expensive route to D; G is then selected.	Goal reached

Path Comparison

Path	Total Cost
S → G	12
S → A → C → G	1 + 1 + 2 = 4
S → A → C → D → G	1 + 1 + 1 + 3 = 6
S → A → B → D → G	1 + 3 + 3 + 3 = 10
Minimum-cost path: S → A → C → G Total cost = 4	

UCS Pseudocode

```

FUNCTION UniformCostSearch(start, goal)
    priorityQueue ← empty priority queue
    INSERT (start, cost = 0, path = [start])

    bestCost[start] ← 0

    WHILE priorityQueue is not empty DO
        current ← REMOVE node with lowest cost

        IF current.node = goal THEN

```

```
    RETURN current.path, current.cost
END IF

FOR each neighbour of current.node DO
    newCost ← current.cost +
        EdgeCost(current.node, neighbour)

    IF neighbour is not in bestCost OR
        newCost < bestCost[neighbour] THEN
        bestCost[neighbour] ← newCost
        newPath ← current.path + neighbour
        INSERT (neighbour, newCost, newPath)
            INTO priorityQueue
    END IF
END FOR
END WHILE

RETURN "No path exists"
END FUNCTION
```

Interpretation of Result

The direct route $S \rightarrow G$ costs 12 units, while the UCS-selected route costs only 4 units.

$$\text{Cost saving} = 12 - 4 = 8 \text{ units}$$

$$\text{Percentage reduction} = ((12 - 4) / 12) \times 100 = 66.67\%$$

Therefore, the logistics company should use $S \rightarrow A \rightarrow C \rightarrow G$. This route reduces the cost by approximately 66.67% compared with the direct route.

Overall Conclusion

- The Water Jug problem demonstrates state-space formulation and legal operators.
- The Mars Rover is best modelled using a hybrid intelligent-agent architecture.
- The Eight-Queens problem is efficiently solved using depth-first search with backtracking.
- OLA cab booking uses goal-based and utility-based reasoning to compare alternatives.
- Uniform Cost Search identifies $S \rightarrow A \rightarrow C \rightarrow G$ as the optimal logistics route with cost 4.